

User guide for snapshot STATE ESTIMATION (SENSEI)

5 de diciembre de 2025

Índice

1. Operating scheme	2
1.1. Differences with calibration and multiperiod SE	3
2. Input .inp file info_File containing the initial information	3
2.1. Time parameters	3
2.2. Demand patterns	3
2.3. Piezometric head (reservoirs)	4
2.4. Initial status of control elements	4
3. .csv files for reading sensor measurements and/or pseudomeasurements	4
4. Aspects that affect state estimation	5
4.1. Measurements / pseudomeasurements and weights	5
4.2. Jacobian	5
4.3. Algorithm for instantaneous state estimation	6
4.4. Default values for the algorithm parameters	7
4.5. Control of parameter increments	7
5. Configuration .txt file (config_snapshot_SE.txt)	8
6. .csv file with the SE results	9

1. Operating scheme

This document explains how to configure the **instantaneous (*snapshot*) state estimation (SE)** tool of *SENSEI*. This is the most basic version of state estimation, applied to a network with point measurements collected at a single time instant, and performing instantaneous simulations (i.e., simulations that do not extend over time). It is intended as a demonstration tool, mainly designed to analyse simple cases and directly observe how adding, removing, or modifying measurements affects the adjustment. Some details in the explanation refer to the estimation task carried out on the network *Red1*, composed of one tank T1, followed by pipe P1, a first demand node J1, a second pipe P2, and a final demand node J2.

We detail how to include measurement/pseudomeasurement `.csv` files and other aspects needed for the correct operation of the tool. All required files are located in the folder ***Snapshot_SE***. The user may use the provided files and even change the values in the `.csv` files, modify measurements, etc., to observe how these changes affect the adjustment process.

The folder ***Snapshot_SE*** is structured as follows:

- **Executable** file `Snapshot_SE_SENSEI.exe`
- **Configuration** file `config_snapshot_SE.txt`
- Subfolder ***Examples_Red1***, which contains:
 - **info_File**, named `Red1.inp` (**network model**)
 - Subfolder ***Example1***: observable scenario in which $q(J2)$, $p(J2)$ and $Q(P1)$ are measured with exact (noise-free) values with respect to the “true” situation.
 - Subfolder ***Example2***: same scenario but with some noise and a different configuration of CVs.
 - Each of the two subfolders includes the measurement and/or pseudomeasurement files **dm_File**, **fm_File**, **pm_File** and **wlm_File**, with their corresponding coefficients of variation.
- **13 dynamic libraries**: `epanet2.dll`, `libamd.dll`, `libcamd.dll`, `libccolamd.dll`, `libcholmod.dll`, `libcolamd.dll`, `libgcc_s_seh-1.dll`, `libgfortran-5.dll`, `libgomp-1.dll`, `libopenblas.dll`, `libquadmath-0.dll`, `libsuitesparseconfig.dll` and `libwinpthread-1.dll`

The executable `Snapshot_SE_SENSEI.exe` opens a terminal from which the user will see the progress of the instantaneous SE, executed automatically according to the predefined configuration. The user only needs to proceed through the pauses between steps by pressing any key when indicated.

Below we detail the modifications that the user is allowed or not allowed to make in order for SE to run properly.

What should the user NOT do?

- Delete any of the dynamic libraries (`.dll` files)
- Change the name of the configuration file `config_snapshot_SE.txt`, since the executable was built to read a file with exactly that name.
- Rearrange, or, in general, modify the internal structure of the folder ***Snapshot_SE***, since this may cause issues related to directories and file locations.

What the user MAY modify

- Modify the content of the measurement or pseudomeasurement files.

- Change the name of any file within each subfolder (including the subfolder itself), provided that the new name is updated in the configuration file `config_snapshot_SE.txt`.
- Change algorithm-related parameters within `config_snapshot_SE.txt`, although it is recommended to keep the default values.
- Select English or Spanish as the language for displaying information and messages during the process.

Nevertheless, **any modification must always follow the guidelines indicated later** in each specific section.

1.1. Differences with calibration and multiperiod SE

Of the three tools provided by SENSEI (calibration, pseudo-dynamic SE, and instantaneous SE), this is the simplest and the one requiring the least configuration. The goal of this type of SE is, given a calibrated model, to find the **most likely value of the state variables (tank levels and demands at demand nodes) at a specific instant** given a set of measurements and pseudomeasurements corresponding to that instant.

As mentioned earlier, and as its name suggests, the simulations performed during the adjustment do not extend over time; they are instantaneous. For this reason, the convergence of the algorithm is much faster and no sophisticated adjustment tools are needed.

However, the applications of this type of SE are more limited than those of its multiperiod (sequential and pseudo-dynamic) version. Due to its instantaneous nature, this SE is used mainly for illustrative purposes rather than for detecting anomalies that may persist over time. Moreover, it does not allow one to exploit capabilities that are exclusive to the multiperiod SE, such as tracking the evolution of control-element status or estimating tank levels at the end of each interval with subsequent uncertainty propagation and correction of initial tank level measurements for the next interval. All these features rely on simulating over a time window, which is not carried out in instantaneous SE.

2. Input `.inp` file `info_File` containing the initial information

The `.inp` file referred to as `info_File` will serve as the **reference model for state estimation**, as it is used to store information related to the network **topology**, **count the number of elements** of each type, and **store key positions** of certain elements.

→ For more information on how to modify an `.inp` file in a way compatible with EPANET, **you can consult this [website](#)**.

2.1. Time parameters

Unlike in multiperiod SE and in calibration (where the [TIMES] section of `info_File` is used to configure simulation duration and the sizes of vectors, matrices, etc.) in instantaneous SE whatever is written in this section of `info_File` is irrelevant, since **the simulation duration is automatically set to 0 (instantaneous simulation)**.

2.2. Demand patterns

For snapshot SE, there is no restriction regarding the length of demand patterns. For any demand pattern in `info_File`, its length is reduced to 1 and its (single) entry is set to 1. In this way, since we work at a single instant, the net demand is modelled just through the base demand, ignoring the multiplicative factor, which has no effect when set to 1.

2.3. Piezometric head (reservoirs)

In SE, the piezometric head of a reservoir is not conceived as a variable to be optimized. It is treated as a fixed value, specified by the user in `info_File`.

If one wishes to “question” the piezometric head of a reservoir ,i.e., to consider it as an optimizable variable; it would need to be modelled as a tank.

2.4. Initial status of control elements

The status of pipes, pumps, and valves used in the simulation will be exactly what is specified in `info_File`.

3. .csv files for reading sensor measurements and/or pseudomeasurements

These files contain the information collected by the sensors for:

- Demand at **all** demand nodes, in `dm_File`.
- Water level in **all** tanks (reservoirs are NOT included), in `wlm_File`.
- Pressure at junctions, in `pm_File`.
- Instantaneous flow, for those links equipped with sensors that measure flow at specific instants, in `fm_File`.

→ **Averaged-flow measurements** (for links whose sensors measure flow based on accumulated volume over a time interval) **are not considered in this type of SE**, since the process is instantaneous. Therefore, in this instantaneous version of SE, no `fm_avg_File` is defined.

Each file must be structured as follows:

Structure of the sensor-measurement .csv files

- **Each row** corresponds to **one sensor**.
- Regarding the **columns (there are 3)**:
 - 1st column: **ID of the element on which the sensor is installed**, which must match an element in `info_File`.
 - For `dm_File`, the ID must match a demand junction (those with nonzero base demand).
 - For `wlm_File`, the ID must match a tank.
 - For `pm_File`, the ID must match a junction.
 - For `fm_File`, the ID must match a link (pipes, pumps, or valves).
 - 2nd column: **coefficient of variation (CV)** or measurement error rate. It represents the percentage deviation from the true value, expressed as a number between 0 and 1.
 - 3rd column: **measured value** at the instant at which the SE is performed.

For example:

	A	B	C
1	T1	0.01	3
2	T2	0.02	0.5

In the example above, the measurements of 2 level sensors located in tanks T1 and T2 are shown for a single instant. Each of them has a different CV.

Optional and mandatory files

- The files `pm_File` and `fm_File` are **optional**. If no sensors of those types are available, there is no need to provide an empty `.csv` file.
- The files `dm_File` and `wlm_File` are **mandatory**. The reason is that, being **state variables**, EPANET requires a value for them in order to run a simulation.
 - If no sensor values are available for some demand or some tank level, a **pseudomeasurement** must be provided. This is simply a value based on a rough estimate, belief, or historical record, and should be assigned a relatively high CV (e.g., between 0.1 and 0.3).
 - When reading `dm_File` and `wlm_File`, it is checked that the number of rows matches the number of demand nodes and tanks in the network, respectively. Otherwise, the user is required to correct the `.csv` file.

4. Aspects that affect state estimation

This section provides a summarized description of how certain components of SENSEI's pseudo-dynamic state estimation process work, giving the user a general idea of how choosing specific parameters or *inputs* may influence the final result of the estimation. It is also indicated how some of these values can be modified through the configuration file.

4.1. Measurements / pseudomeasurements and weights

Reading the measurement and pseudomeasurement `.csv` files provides, for each defined element and for each instant, a measurement z_i with an associated coefficient of variation CV_i . This allows us to define the standard deviation of the measurement / pseudomeasurement as $CV_i \cdot z_i$. The weight corresponding to z_i , denoted by w_i , is defined as the inverse of the measurement variance:

$$w_i = \frac{1}{(CV_i \cdot z_i)^2} \quad (1)$$

Notes on weights

- The lower the variability of a measurement, the greater our confidence in it and the larger its weight, and vice versa. The weight (relative to the measured value) reflects the *importance* that the variable will have in the objective function.
- If $z_i = 0$ (the measurement is zero; for example, a flow measurement equal to 0 because no water is circulating through the pipe), then w_i is automatically set to 0.05 to avoid division by zero in (1).

4.2. Jacobian

The Jacobian matrix is the key component of the optimization algorithm. It contains the derivatives of the measurements (*rows*) with respect to the state variables (*columns*).

Numerical computation of the Jacobian

SENSEI computes all derivatives numerically:

- At each iteration, we start from the vector $g(\mathbf{x})$, which contains the simulated values corresponding to the measurements / pseudomeasurements for a given set of state variables $\mathbf{x} = (x_1, \dots, x_n)$.
- The variable with respect to which the derivative is to be computed (assume it is the j -th variable) is perturbed by $\varepsilon\%$, yielding the perturbed vector $\mathbf{x}_\varepsilon = (x_1, \dots, x_j(1 + \varepsilon/100), \dots, x_n)$, and the system is simulated again to obtain $g(\mathbf{x}_\varepsilon)$.
- The derivative vector (the entire j -th column) is then computed as

$$\frac{g(\mathbf{x}_\varepsilon) - g(\mathbf{x})}{x_j \cdot \varepsilon/100} \quad (2)$$

→ For the derivative computation in (2) to reflect the real model behavior, the perturbation must be *small*. Therefore, the default perturbation rate is $\varepsilon = 0.2$. Later we will explain how to modify it.

4.3. Algorithm for instantaneous state estimation

The goal of the SE process is to find the most probable values of the state variables (demands at demand junctions and tank levels) given the available information, solving a nonlinear least-squares problem.

Since this version of SE is designed for illustrative examples based on relatively small networks, the problem is compact and manageable. For this reason, we use an algorithm based on **Gauss–Newton**, unlike calibration and multiperiod SE, which rely on Levenberg–Marquardt.

In the search for the optimal state variables, we iterate up to `max_it` times. In each iteration, we solve the **equation**

$$(J_k^t \cdot W \cdot J_k) \cdot \Delta \mathbf{x}_{k+1} = J_k^t \cdot W \cdot (z - g(\mathbf{x}_k)) \quad (3)$$

Pseudocode of the algorithm

```

1 x0 ← initial state of the network #initial guess for the state variables
2 Compute residuals r = z - g(x0)
3 F(x0) = r^t*W*r #initial objective function value
4 SWSE ← F(x0)
5
6 for k in 1:max_it:
7     break_while ← 0
8     x_k ← state in iteration k
9
10    while break_while == 0:
11
12        Compute J(x_k)
13
14        Solve: (J^t*W*J) Δx = J^t*W*r
15        x_prov ← x_k + Δx
16        Compute residuals r = z - g(x_prov)
17        F(x_prov) = r^t*W*r #objective function for provisional variables

```

```

18     SWSE_prov ← F(x_prov)
19
20     if SWSE_prov < SWSE:
21         x_{k+1} ← x_prov    #update variables if improvement is
                             achieved
22         SWSE ← SWSE_prov
23
24         if convergence:
25             break_while ← 1
26     else:
27         break_while ← 1 #if no improvement, terminate using the last
                             update

```

Note: $SWSE = \text{Sum of Weighted Squared Errors}$

Let $\mathbf{x}_{\text{new}}$ and $\mathbf{x}_{\text{old}}$ denote the newly updated variables and the variables from the previous iteration, respectively. Convergence is assessed through the rate

$$\frac{\|g(\mathbf{x}_{\text{new}}) - g(\mathbf{x}_{\text{old}})\|_1}{\|g(\mathbf{x}_{\text{new}})\|_1}$$

If this rate is smaller than the tolerance `tol` set at the beginning, the algorithm is considered to have converged and the process ends.

4.4. Default values for the algorithm parameters

Default values for the algorithm parameters

- Perturbation rate for the Jacobian computation: $\varepsilon = 0.2$
- Maximum number of iterations: `max_it` = 5
- Tolerance: `tol` = $3\text{e-}4 = 0.0003$

→ The values of these algorithm parameters can be modified in the configuration file (explained in a later section), although **it is recommended to keep the default values**.

4.5. Control of parameter increments

Solving (3) provides an increment of the state variables, $\Delta\mathbf{x}_{k+1}$. The provisional variables are computed by adding this increment to the current variables:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}_{k+1} \quad (4)$$

However, (4) may lead to undesired or physically impossible values:

- For tank levels, if the new estimated value exceeds the maximum level or falls below the minimum, it will be bounded accordingly.
- For demands, negative estimated demands are not allowed. In such cases, the value is clipped to 0.

5. Configuration .txt file (config_snapshot_SE.txt)

In this file, named `config_snapshot_SE.txt` and **whose name cannot be changed**, the user must specify the language in which information and messages will be displayed, the name and location of the files that will be used for SE, and the values of the algorithm hyperparameters. An example is:

```
1 LANGUAGE (ENG/ESP); ENG
2 -----
3 PATH; Examples_Red1
4 -----
5 info_File; Red1.inp
6 dm_File; Example1/dm_Red1.csv
7 wlm_File; Example1/wlm_Red1.csv
8 pm_File; Example1/pm_Red1.csv
9 fm_File; Example1/fm_Red1.csv
10 -----
11 max_it; -
12 epsilon; -
13 tol; -
14 emitter_ID; -
```

Considerations regarding the .txt configuration file

- The **language** may be **English or Spanish**, activated by writing '**ENG**' or '**ESP**', respectively, in the field LANGUAGE. If the value is left empty or anything else is written, English will be activated by default.
- The measurement / pseudomeasurement .csv files and the .inp info_File must be located in a **subfolder**, specified in PATH.
- It is **mandatory to specify a calibrated reference model** info_File, as well as a **measurement / pseudomeasurement file for the demands** at demand junctions (dm_File) and for the **tank levels** (wlm_File).
- pm_File and fm_File are **not required** if such measurements are not available. In that case, the field must contain a ' - ' (a **dash**).
- To keep the **default values of the hyperparameters** of the algorithm, simply write ' - ' (a **dash**), as shown in the example. If any value is to be modified, it must be explicitly written.
- The field emitter_ID must remain as shown above (a dash) if one wishes to perform SE using the model exactly as it appears in info_File, without additional leaks beyond those possibly present in the inp. **If, however, after performing SE, evidence of a leak appears in a particular area (based on normalized residuals), SENSEI allows performing a new SE in which the emitter coefficient at a chosen node is estimated jointly with the state variables.**
- **IMPORTANT: Do not modify** the text on the left-hand side (before the ' ; '), since the executable is programmed to read it exactly as written in order to store the information correctly.

→ If within the folder specified by PATH (in our case, *Examples_Red1*) there exist subfolders that contain the .csv files dm_File, wlm_File, pm_File, and fm_File (as is the case of the sub-subfolders *Example1* and *Example2*), the internal path of each file must also be specified inside PATH; for example, *Example1/dm_Red1.csv*, *Example1/wlm_Red1.csv*, etc.

In short, the correct path to both the `.csv` files and the `.inp` file must always be provided, including any subfolders inside the main `PATH` folder.

6. `.csv` file with the SE results

At the end of the adjustment process, the most relevant information from the SE is stored in a single `.csv` file, structured as follows:

Structure of the SE results `.csv` file

- **Each row** corresponds to **one variable**.
- Regarding the **columns**, in addition to the one indicating the variable name, we have:
 - **Measured value** (the value provided in the `.csv` files as a measurement or pseudomeasurement)
 - **Measurement CV** (the coefficient of variation assigned to each value in the `.csv` files)
 - **Estimated value** obtained through SE
 - **Posterior CV** of the estimation
 - **Normalized residual**

The resulting file is saved inside `PATH` (in our case, `Examples_Red1`). Furthermore, to avoid overwriting results from previous SE runs, the file name includes the date and time at which it was saved, in the formats `YYYYMMDD` and `hhmmss`, respectively. Thus, assuming that `info_File = Red1.inp` and that the file was saved on 24/12/2025 at 21:35:42, the output file will be `Red1_results_20251224_213542.csv`